Python Basics and Programming Concepts

From simple concepts to powerful programs!



1. Introducing Python

- What is Python?
- Why use Python?
- Setting up Python
- Writing your first program

```
print("Hello, Python!")
```

Print statement

```
print("Hello, Python!")
Hello, Python!
```

2. Types and Operations

Variables & data types

Numbers
```
a = 10
b = 3.5
```

Strings
```
name = "Python"
print(name)
```

Lists
```
nums = [1, 2, 3]
nums[0]
```

Tuples
```
t = (1, 2, 3)
t[0]
```

Dictionaries
```
d = {"a":1, "b":2}
d["a"]
```

Files
```
with open("file.txt")
  as f:
    print(f.read())
```

Numeric Types
```
int, float, complex
```

Dynamic Typing
```
x = 5
x = "hello"
```

3. Statements and Syntax

- Assignments
- Expressions
- Statements
- Loops and Iterations
- Comprehensions

```
x = 10
y = x + 5
if x > 5:
    print("Greater")
for i in range(5):
    print(i)
```

If condition / for loop

Simple syntax = powerful ts!

4. Functions

- Function Basics
- Scopes
- Arguments
- Advanced Functions

```
def greet(name):
    return f"Hello, {name}!"
print(greet("Python"))
```

Return Values | Parameters | Default Arguments | *args / **kwargs

Write once, use many times!

5. Modules

- Module Coding Basics
- Module Packages
- Advanced Module Topics

Importing a module

```
import math
print(math.sqrt(16))
```

```
4.0
```

module.py → import module

- os
- sys
- math
- random
- ...

Packages (collection of modules)

Use existing modules or create your own!

6. Classes and OOP

- Class
- Operator Overloading
- Class Designing

Class, object & methods

```
Class Car:
    def __init__(self, brand):
        self.brand = brand
    def show(self):
        return f"Car: {self.brand}"
c = Car("Toyota")
print(c.show())
```

Class → Object → Attributes & Methods

Build real-world models!

7. Exceptions and Tools

- Exception Basics
- Exception Coding Details
- Exception Objects
- Designing With Exceptions
- Parallel System Tools

Try-except error handling

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
```

Tools & Libraries
- threading
- multiprocessing
- concurrent.futures
- queue

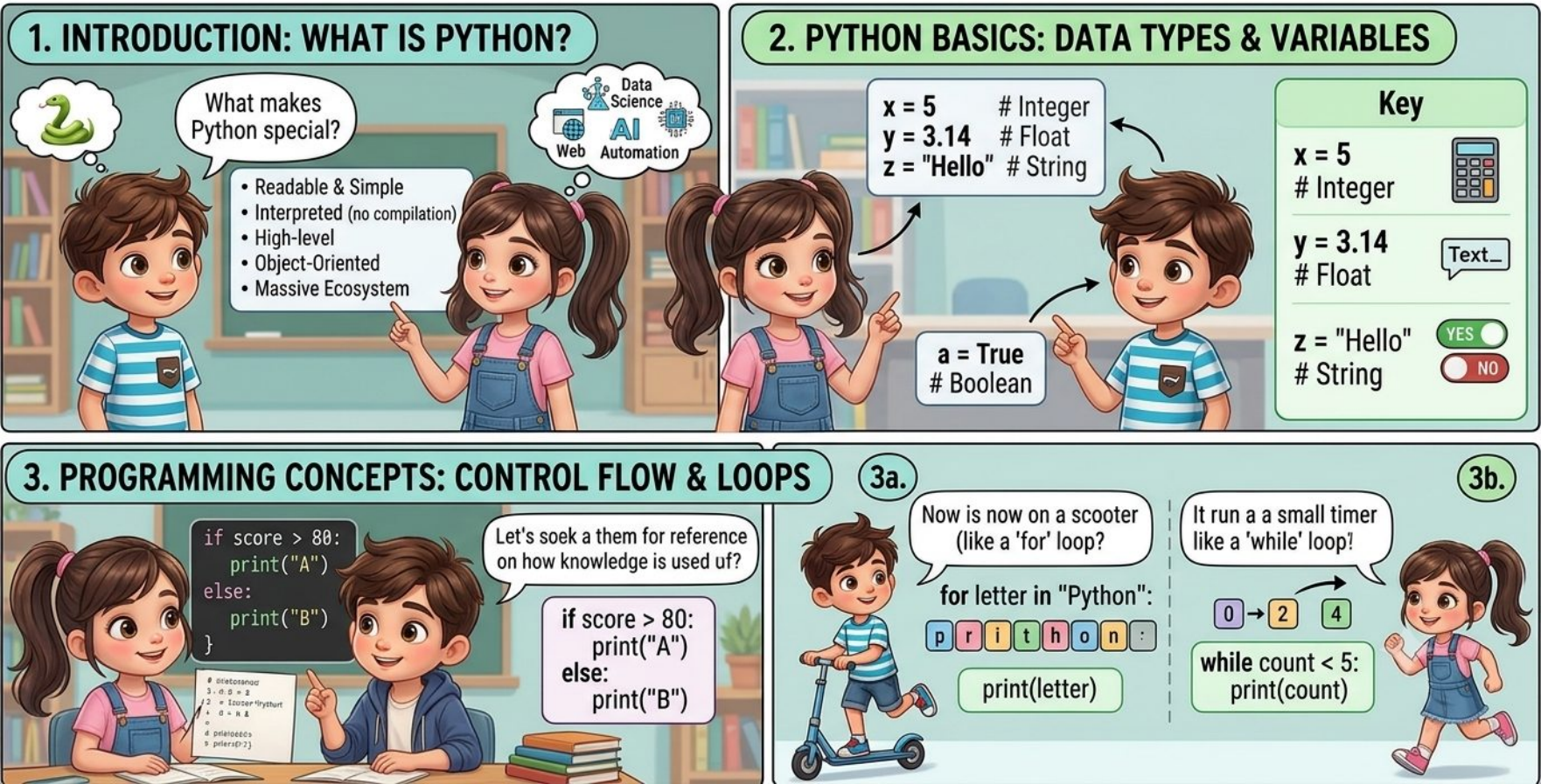
Handle errors, manage resources, use system tools for better performance!

Write → Run → Build → Grow

Code block = a group of code lines that perform a task.

Python + Logic + Creativity + Endless Possibilities

PYTHON BASICS AND PROGRAMMING CONCEPTS: INTRODUCING PYTHON



PYTHON DATA TYPES AND OPERATIONS: NUMBERS, STRINGS, LISTS, TUPLES, DICTIONARIES

1. NUMBERS & ARITHMETIC

```
x = 10      # Integer
y = 3.14    # Float
```

Math is easy! Try some basic math or special ones.

- 5 + 3 = 8
- 10 / 2 = 5.0
- 15 % 4 = 3 (modulo)
- 2 ** 3 = 8 (power)

Key

x = 10 (int)
y = 3.14 (float)

2. STRINGS & BASIC TEXT OPS

```
name = "Maya"
greeting = 'Hello'
```

```
print("Hi " + name)      Concatenation
print(name.upper())      Methods
print(len(name))         Length
```

A text block can be anything we say or type!

Key

name = "Maya" (str)

3. COLLECTING THINGS: LISTS & TUPLES

3a. LISTS (Ordered, Changeable)

```
fruits = ["🍎", "🍌", "🍇"]
```

We can change them! Add, remove, sort.

indexed view! [0], [1], [2]

3b. TUPLES (Ordered, Unchangeable)

```
coordinates = (40.7, 74.0)
```

Once set, they're frozen! Good for fixed data.

Tuple is fixed! 🔒 (...)

4. KEY-VALUE PAIRS: DICTIONARIES

```
pet = {"name": "Buddy", "age": 2}
```

Data

Look-up

Look up data with special keys!

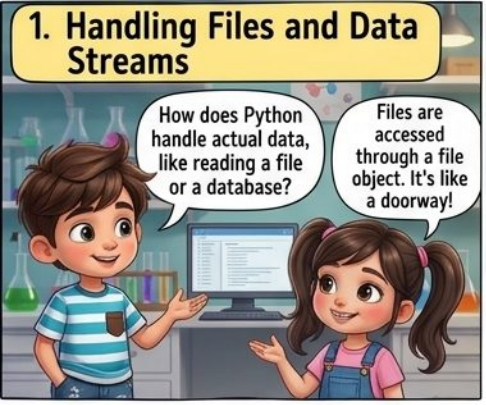
Key	Value
"name" →	"Buddy"
"age" →	2

print(pet["name"]) → Retrieval

pet["age"] = 3 → Updating

It's like an index!

1. Handling Files and Data Streams



Text Files (.txt, .csv)



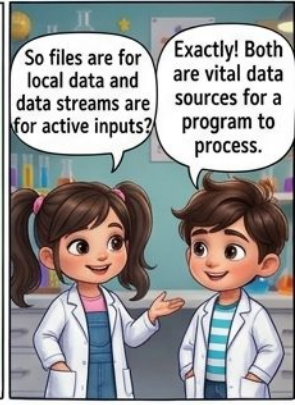

```
with open('data.txt', 'r') as f:
    print(f.read())
```

Binary Files (.jpg, .mp4)

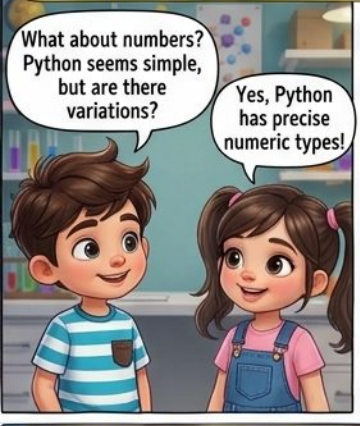


Binary Files (.jpg, .mp4)

```
with open('image.jpg', 'rb') as f:
    byte_data = f.read()
```



2. Exploring Python's Numeric Types



integers (int)

- Whole numbers, unlimited precision.

```
42
-10
```

floating-point numbers (float)

- Decimal values, standard precision.

```
3.14159
```

complex numbers (complex)

- For math and science.

```
2 + 3j
```

Decimal numbers (Decimal)

- Fixed precision, for currency.

```
from decimal import Decimal
Decimal('1.00')
```

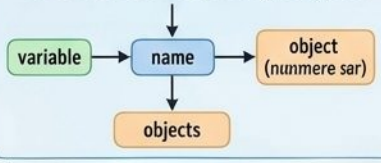


3. Understanding Dynamic Typing and the object model



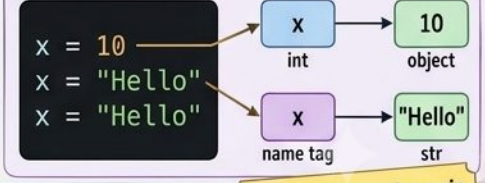
Variable as Name Tag (References)

Variables are *names* pointing to *objects*.



Dynamic Typing (Runtime Type Check)

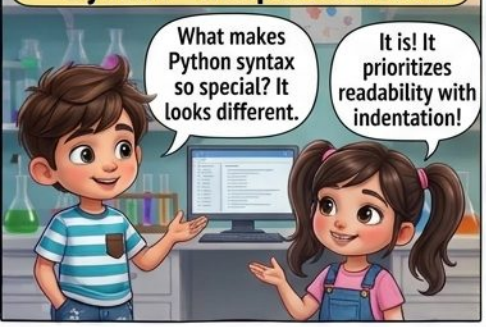
```
x = 10
x = "Hello"
x = "Hello"
```



So there's no fixed type for a variable.
We decide what the variable refers to dynamically...

right objects + dynamic typing = powerful and flexible code! 🙂

1. Understanding Python Syntax – Simple and Clean



Python Syntax Rules

Indentation is Scope

- Indentation is a Scope to first indentation.

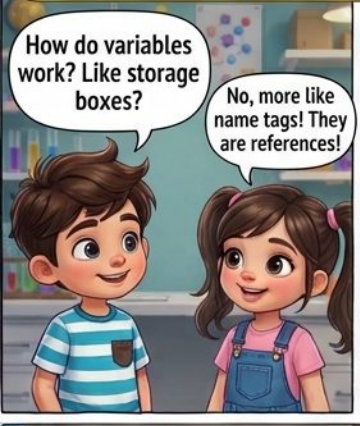
```
if x > 5:
    print("Big")
```

No semicolons or braces needed

- No semicolons are or braces needed.
- No semicolons or braces needed.



2. Master Assignments and Variables – Creating Objects



Variable Assignments (Binding)

`x = 10` → `10` (object)

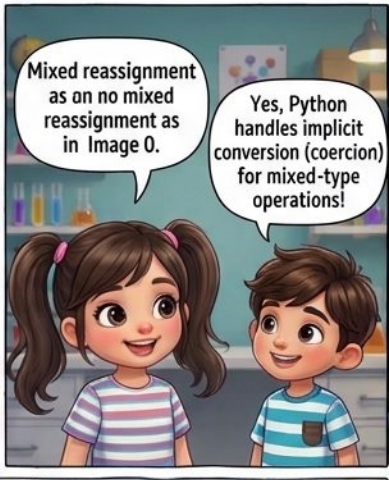
```
x = 10
y = x
print(id(x), id(y))
```

`x = "Hello"` → `"Hello"` (str)

```
x = 10
x = "Hello"
print(id(x), id(y))
```

`x = [1, 2]` → `[1, 2]` (list)

```
x = [1, 2]
y = [1, 2]
print(id(x), id(y))
```

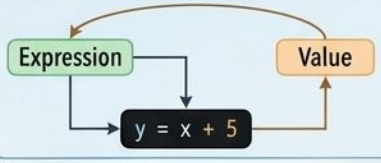


3. Unpacking Expressions and Statements – The building blocks

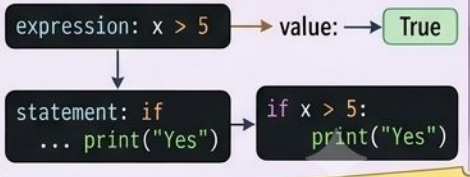


Variable as Reference

Evaluates and *ıames* pointing to *objects*.



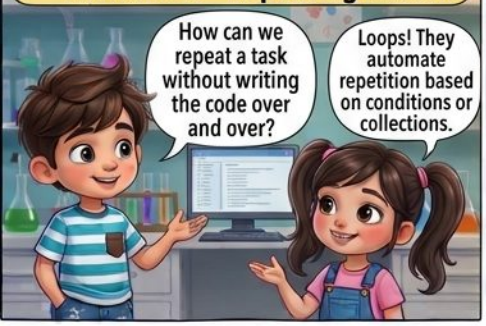
Expression vs Statement flow



So every expression produces a value, but not every statement does. Assignments and print are statements.

Right structure + variable references = powerful and flexible code! ☺

1. Mastering Loops & Iterations – Repeating Actions



Python Advanced Concepts – Efficient Code

for Loops

for Loop

for iterates over a collection (list, string, range).

```
colors = ['red', 'green']
for color in colors:
    print(color)
```

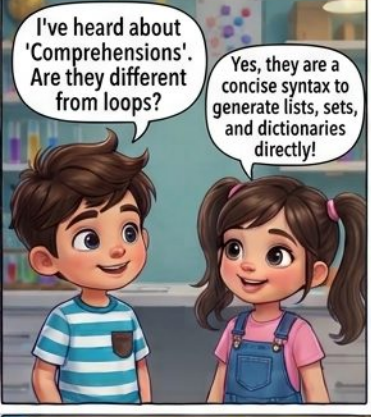
while Loop

while repeats as long as a condition is true.

```
count = 0
while count < 2:
    print(count)
    count += 1
```



2. The Power of Comprehensions – Creating Collections Quickly



List Comprehension (Syntax)

[new_item for item in old_collection if condition]

List Comprehension (Binding)

```
squares = []
for x in range(3):
    squares.append(x**2)
print(squares)
```

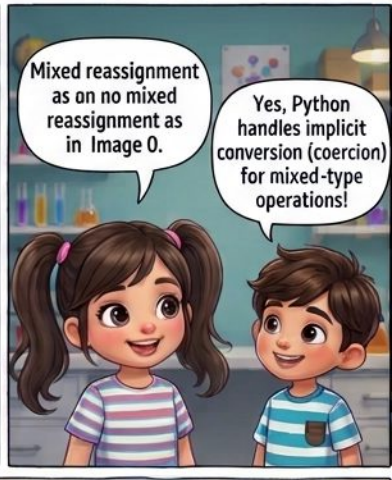
```
squares = [x**2
    for x in range(3)]
print(squares)
```

Set Comprehension

```
{x**2 for x in range(3)}
```

Dictionary Comprehension

```
{x: x**2 for x in range(3)}
```

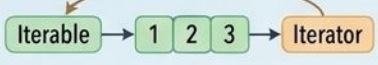


3. Unpacking Iterations & Iterables – The Engine of Loops



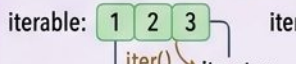
Iterable vs Iterator

Evaluates and *iames* pointing to *objects*.



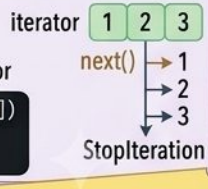
**Iterable*: Object capable of returning items one-by-one (list, range).

Iter Protocol Flow




```
my_iter = iter([1, 2, 3])
next(my_iter) # 1
next(my_iter) # 2
```

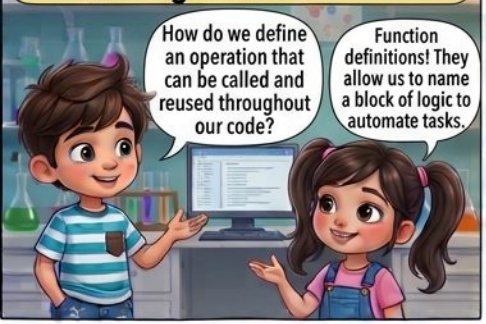
Iterator Pointer



So, an iterable is the 'what' and an iterator is the 'how' that a loop uses. List comprehensions wrap this logic into one clean expression.

Efficient loops + comprehensions = powerful and elegant code! 😊

1. Mastering Function Basics – Defining Actions



Python Advanced Concepts – Powerful Reusable Code

Function Definition Syntax

def function_name(parameter):

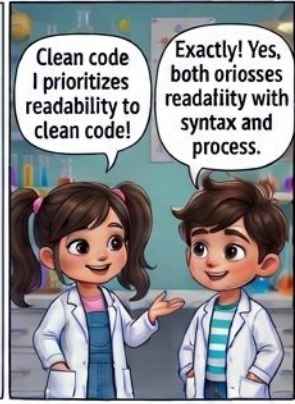
def is the keyword.
Parameters receive inputs.
return is used to send values back.

print(function_name(value)) is an example of a call.
Calls pass **arguments**.

Calling a function:

```
# Definition
def greet(name):
  return "Hello, " + name

# Calling
greeting = greet("World")
print(greeting)
```



2. The Power of Scopes – Variables and Namespaces



Scopes & Name Lookups (LEGB)

LEGB Rule

Local: Arguments level.
Enclosing (nested): module-level)
Global: (module-level)
Built-in: (pre-defined)

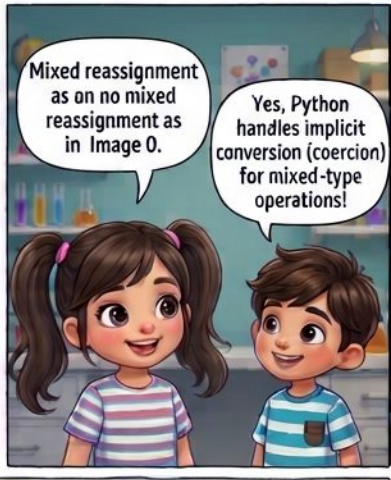
```
# Scope Demo
x = "global"  # G
def outer():
  x = "enclosing"  # E
  def inner():
    x = "local"  # L
    print(x) # Finds 'local'
  print(x) # Finds 'enclosing'

outer()
print(x) # Finds 'global'
```

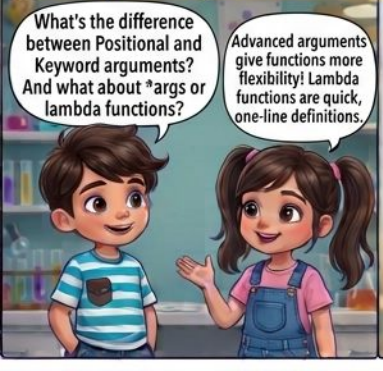
```
# Scope Demo
x = "global"  # G
def outer():
  x = "enclosing" # E
  def inner():
    print(x)
  inner()

outer()
print(x) # Finds 'global'
```

```
# global keyword
count = 0
def increment():
  global count
  count += 1
```



3. Unpacking Advanced Arguments & Functions – Flexible Tools



Argument	Description	
Positional	Arguments passed in order.	`greet("A")`
Keyword	Passed by parameter name.	`greet(name="B")`
Default	Optional; uses value if argument omitted.	`def greet(name="User")`
*args, **kwargs	Collects extra positional/ keyword arguments.	`def func(*args)`

Simple vs Lambda Function

function input → function output → `# Tradttional def def square(x): return x**2`

def vs lambda x: x**2

`# Anonymous Lambda lambda_square = lambda x: x**2`

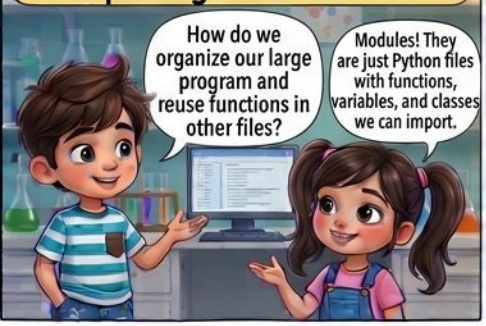
```
# Advanced Arguments Demo
def advanced_greet(greeting, *names):
  for nuoe 2n naees:
    prtnt(greeting, name)
advanced_greet("Hi", "A", "B")
```

```
print(square(2))
print(lambda_square(2))
```

So, functions wrap logic. LEGB defines name search, and advanced arguments allow for more generic and reusable functions.
Lambda is just an inline shortcut!

Flexible arguments + custom scopes = powerful and modifiable code! 😊

1. Mastering Module Basics – Importing Code



Python Advanced Concepts – Powerful Reusable Code

Module Import Syntax

'import module_name'

import is the keyword.
Code is loaded and executed.

'from module_name import thing'

*Specifically imports variables/functions.
Access them directly without module name prefix.

Calling a function:

```
# Module definition:
math_ops.py
def square(u):
    return x*x

# Calling file: main.py
import math_ops
result = math_ops.square(4)
print(result)
```



2. The Power of Packages – Complex Structures



Module Packages and Namespace Structuring

```
main_pkg/
  __init__.py
  main_pkg/
    __init__.py
  sub_pkg/
    __init__.py
  sub_pkg/
    __init__.py
  utils.py
```

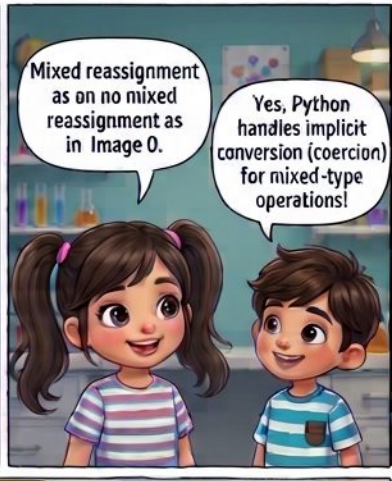
```
# main.py inside same folder
import pkg
print(pkg.multiply(2, 3))
```

```
# __init__.py inside pkg/
# Exposes math_utils.py content
from .math_utils import *

# main.py inside same folder
import pkg
print(pkg.multiply(2, 3))
```

LEGB Rule

```
x = "global" # G
def global()
  x = "enclosing" # E
  def inner():
    onter(9)
  inner()
outer()
print(v) # Finds 'global'
```

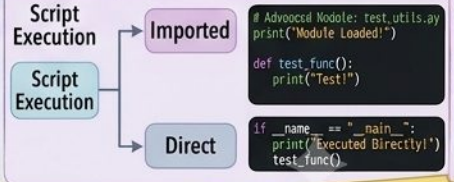


3. Unpacking Advanced Module Topics – Search Paths & Execution



Aspect	Description
sys.path	A list of directories Python searches for modules. `import sysz print(sys.path)`
__name__	A special built-in variable for execution context.
"__main__"	The script being executed directly.
Relative Import	Importing from same package structure. `from .subpkg import thing`

Import Path and Execution



So, modules structure code. Packages organize collections, and advanced topics control name lookup and direct execution. '__name__' is key!

Module reusability + Package organization = powerful and modifiable code! 😊

1. Mastering Class Basics – Blueprint for Objects



Python Advanced Concepts – Powerful Reusable Code

Class Definition Syntax

class MyClass:

class keyword. Defines attributes and methods.

__init__(self, data):

*The constructor. Initializes an object's attributes. *self* refers to the specific instance.

Calling a function:

```
# Definition
class Rebot:
  def __init__(self, name):
    self.name = name
  def greet(self):
    return "I am + self.name

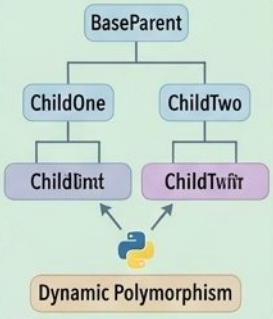
# Instance
my_robot = Robot("Wall-E")
print(my_robot.greet())
```



2. The Power of Class Design – Building Hierarchies



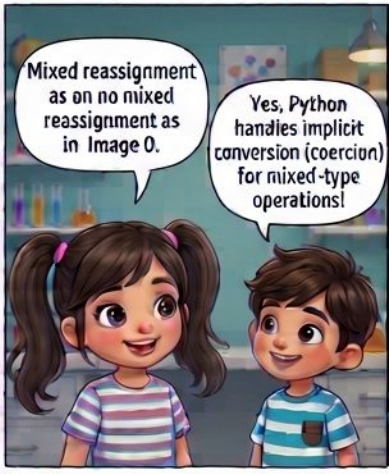
Class Inheritance and Polymorphism




```
# Base Class
class Shape:
  def area(self): pass

# Child Classes
class Circle(Shape):
  def __init__(self, r):
  def area(self):
    return 3.14.r * self.r

class Square(Shape):
  def __init__(self, s):
    self.s = s
  def area(self):
    return self.s * self.s
```

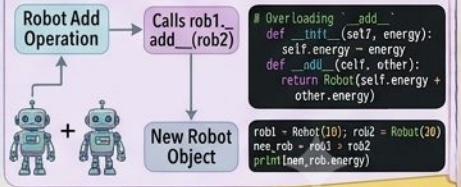


3. Unpacking Operator Overloading – Custom Object Math



Method	Description
__add__	Redefines the '+' operator for addition. `rob1 + rob2`
__str__	Controls string representation. `print(rob1)`
__eq__	Redefines '==' for equality comparison.
__len__	For custom object length.

Special (Dunder) Methods



So, classes encapsulate data. Design builds relationships, and overloading gives our objects natural interactions. Special methods are key!

Class reusability + Design inheritance = powerful and modifiable code!

PYTHON ADVANCED CONCEPTS – ROBUST CODE

1. Mastering Exception Basics – Handling Errors



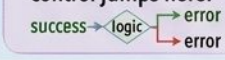
Try / Except Syntax

```
try:
    ... code that might
    ...
```

- Attempts execution. Code inside is 'risky'.

```
except:
    ... handle the error
    ...
```

- If an error occurs, control jumps here.

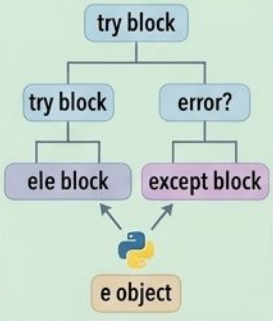




2. Exception Coding Details & Exception Objects



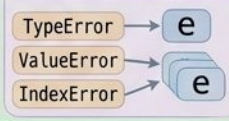
Detailed Flow & Standard Objects



as e Syntax

- Points: Catches the exception *object*. You can inspect `e.args` or print it.

Exception Objects





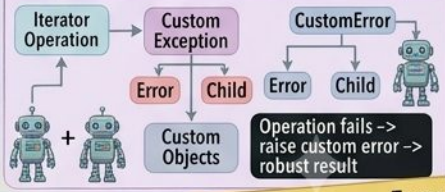
3. Designing With Exceptions – Robust Application Design



Advanced Design and Cleanup

Meth:	Cleanup & Resources
`with`	Points: the `with` statement for `with` statement and ensper finally.
`with`	Points: The exception is enty amendent and itrons.
`finally`	Points: With `with` statement and durabiity and `finally`.

Designing With Exceptions (Dunder Methods & Hierarchies)

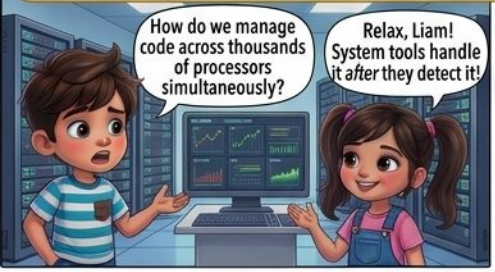


So, basic try-except isolate errors. Details and e objects give context, and a good design with hierarchies handles complex logic. Cleanup is key!

Robust Design + Error handling = powerful and reliable code!

PARALLEL SYSTEM TOOLS – DISTRIBUTED COMPUTING

1. Parallel Framework Basics – Concepts



Distributed Decomposition

Task Decomposition

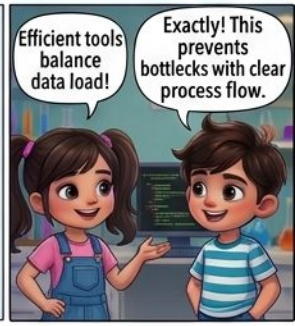
- *Decompose* large problems into sub-tasks (Domain vs. simultaneous execution).

(e.g., Domain decomposition, Functional decomposition)

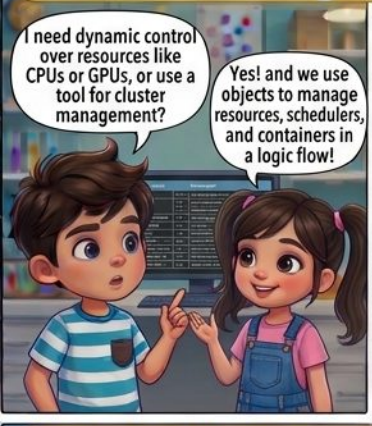
Communication & Load

- *Communicate* results and *Balance* load across nodes (MPI vs. Task Queues).

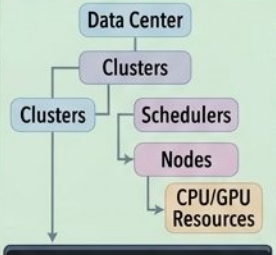
Task Queue → Worker Nodes → Results Database



2. Resource Management & Tool Objects



Cluster Resource Management




```
kubectl apply -f pod.yaml
kubectl apply -
...
```

Resource Objects

Tool Object	Description
KBs Pod	Container resource object
SLURM Job	Scheduler task object
OpenMPI Group	Communication group objest
PyTorch OOP	Distributed Oats Parallel objest

Resource Objects

```
mpirun -np 4 ./my_app
...
```



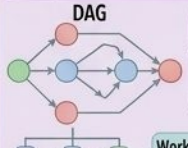
3. Tools & Workflows – Resilient Distributed Design



Resilience and Debugging

Tool Method	Resilience Action
Checkpointing	Periodic state save
Replication	Duplicate critical tasks
Dynamic Scheduling	Re-route on node failure
Heartbeats	Detect node health

Workflow Management
(Tool methods & performance analysis)



Tool	Workflow Tools	
Apache Airflow	Workflow	management
Celery	Management	Distributed task queue
NVIDIA NSight	NSight	Performance profiler
Grafana	Metrics	Metrics visualization

Workflow DAG → Parallel Tool → Performance Profiler → Metrics Dashboard

So, frameworks provide parallel primitives. Management tools handle e-objects and resources, and a good design workflow tools for complex logic and resilience. Monitoring is key!

Scalable tools + efficient management = powerful and reliable parallel systems! 😊